

CSCE 633 MACHINE LEARNING · MODULE 13 · LECTURE NOTES

Deployment

Drift, feedback, and who is affected when it is wrong.

MODULE OUTCOMES

- Explain drift and design monitoring for it.
- Explain feedback loops and why they are the hardest case.
- Treat fairness as a measurement and design problem.
- Explain what a model owes the person it is applied to.
- State what a model can honestly promise.

1 Drift

Kind	What changes	How to detect it
Covariate shift	$P(x)$ — the input distribution moves while the relationship between x and y is unchanged.	Monitor feature distributions. Requires no labels at all , which is why it is the one to build first.
Label shift	$P(y)$ — the base rates move (Module 01 §2).	Monitor the distribution of predictions, which tracks it approximately.
Concept drift	$P(y x)$ — the relationship itself changes. The same input now implies a different outcome.	Requires labels, so it is the hardest to detect and the most dangerous — every feature can look normal while the model is systematically wrong.
Sudden drift	A supplier changes, a policy changes, a sensor is replaced, a competitor launches.	A visible step in the monitored metrics , which makes it the easy case once monitoring exists.
Gradual drift	Slow behavioural or demographic change.	Invisible day to day and obvious over a year — which requires comparing against a fixed reference window rather than against yesterday.

MONITOR THE INPUTS, BECAUSE THE LABELS ARRIVE LATE

Ground truth in production is delayed, partial, or entirely absent. A loan default is known in years; a fraud chargeback in months; a recommendation's true value never. **And the predictions that are never resolved are not a random sample of the predictions made.**

So performance monitoring alone is far too slow. By the time measured accuracy visibly degrades, the model has been making bad decisions for as long as the label delay — which may be the entire period since deployment.

Monitor the input feature distributions, the prediction distribution, and per-feature summary statistics, all of which are available immediately and none of which requires an outcome. Population stability indices, KS statistics, or simple quantile tracking all work.

And alert on change rather than on absolute thresholds. A feature whose mean has moved three standard deviations from its training distribution is worth investigating regardless of whether it crossed a number somebody picked a year ago — **which is the same argument as CSCE 678 Module 07 made about phi accrual failure detection:** adapt the threshold to the observed distribution rather than fixing it in advance.

2 Feedback loops

THE PATTERN

1. model predicts higher risk in region R
2. more inspection / review / attention is sent to R
3. more incidents are FOUND in R -- because more was looked for
4. the new data confirms the model
5. the model becomes more confident about R
6. go to 2

Nothing here is a bug. Every step is the system working as designed. The data is no longer an observation of the world; it is an observation of the system's own behaviour.

OTHER FORMS

recommenders	you only observe clicks on what you SHOWED
hiring	you only see outcomes for who you HIRED
credit	you only see repayment for who you APPROVED
pricing	your price changes the demand you measure

COMMON STRUCTURE: the model's action determines the data it later receives. The distribution is now ENDOGENOUS.

MITIGATIONS: a randomised control slice; log what you did NOT show; explicit exploration; audit against independent data.

WHY FEEDBACK IS THE HARDEST CASE

Module 01 §1's three substitutions assumed the distribution was exogenous — that the world generates data and the model observes it. **A feedback loop breaks that assumption at the root:** the model is now part of the process generating its own training data.

So the usual diagnostics do not fire. The data looks consistent, the model's accuracy on it is high and rising, and every metric agrees — because the model is being evaluated against a world it shaped.

Confirmation is not evidence when you caused it.

And the selection problem compounds. You observe repayment only for approved applicants, performance only for hired candidates, clicks only on shown items — so the model's errors on the cases it rejected are permanently invisible, and it cannot learn that it was wrong to reject them.

Holding out a randomised control slice is the one mitigation that genuinely works — approve a small random sample regardless of the model's recommendation, show a small random selection, inspect a random subset. **It is also the hardest to get approved,** because it means knowingly accepting some bad decisions in exchange for information. **That is a real cost and it buys the only unbiased data you will have,** and making the case for it clearly is a genuine professional skill.

3 Fairness

THE CRITERIA ARE MATHEMATICALLY INCOMPATIBLE

Demographic parity requires equal positive prediction rates across groups. **Equalised odds** requires equal true positive and false positive rates. **Calibration** requires that a given score means the same thing in every group.

These three cannot all hold simultaneously unless the base rates are identical across groups or the classifier is perfect. **This is a theorem** — Kleinberg, Mullainathan and Raghavan; Chouldechova independently — **not a limitation of current methods or a problem awaiting a better algorithm.**

So you must choose which criterion to satisfy, and the choice is a value judgement about what the system is for and who bears the cost of its errors. It cannot be derived from the data, and a technical argument cannot settle it.

What an engineer owes is to make the trade-off explicit: measure all three, report all three, and state which was chosen and on what grounds. **Choosing silently is the failure mode** — and the default behaviour of any system is to choose silently, by optimising aggregate accuracy and letting the criteria fall where they may. **The engineering contribution here is measurement and disclosure, which is squarely within the discipline this whole program has been teaching.**

- **Measure performance by subgroup first** (Module 12 §3), before any intervention. **Most disparities are discovered this way** and many are straightforward data problems — a group under-represented in training, or labelled by a different process.
- **Check the training data's composition against the population the model will actually serve.** A model trained on one population and deployed on another is a covariate shift problem (§1) before it is anything else.
- **Removing the protected attribute does not work.** Proxies reconstruct it — postcode, name, purchase history, device — so the model's behaviour is unchanged, **and removing the attribute makes the disparity impossible to measure. It is the most common response and among the least effective**, and it actively prevents the auditing that would detect the problem.
- **Decide the operating point per group deliberately** if the legal and policy context permits it — which varies by jurisdiction and by domain, and is a question to ask before building rather than after.
- **And document the decision.** A model card stating the intended use, the populations evaluated on, the measured disparities, the criterion chosen, and the known failure modes. **This is the same artifact as CSCE 605's honest compiler claims and CSCE 678's SLO statement** — a written, specific account of what was established.

4 Claiming honestly

- **'AUC 0.84 on data collected in 2023, split by patient, against a majority-class baseline of 0.50.'** The split key and the baseline are what make it a claim rather than a number (Module 01 §2 and §4).

- **'Worst subgroup AUC 0.71, n = 340.'** The slice and its count, so a reader can judge both the disparity and the precision of the estimate (Module 12 §3).
- **'Calibrated on the validation set; nominal 90% intervals empirically cover 88% on test.'** The measured coverage, not the assumed one (Module 10 §4).
- **'Performance degrades measurably when the imaging equipment changes; the retrain trigger is a population stability index above 0.2 on the top five features.'** Stating the known failure mode and the response to it is more useful than any additional point of accuracy.
- **And what you cannot honestly say: 'the model is 94% accurate'** with no baseline, no split key, no slice breakdown, and no date. **Every course in this program ends on this point** — CSCE 735 on speedups, CSCE 620 on robustness, CSCE 605 on miscompiles, CSCE 678 on availability — **and the repetition is the argument.**

WHERE THIS LEAVES YOU

You can fit the standard model families, diagnose which error term is limiting you, choose a metric from the costs of the errors rather than from habit, and design an evaluation that is difficult to fool.

And you know the things that distinguish someone who has deployed a model from someone who has trained one: that an unexpectedly good result is usually leakage; that benchmark labels are several percent wrong; that the validation set wears out; that the average describes nobody; and that the distribution does not hold still.

CSCE 669 supplies the optimisation theory underneath all of this — convexity, duality, and why gradient methods converge — **and CSCE 636 replaces the hand-designed features of Module 11 with learned ones**, which changes what is possible without changing anything in this module.

Because the discipline does not change. Baseline first, split by what will be new, report by slice, measure the uncertainty, and never claim more than you measured. That survives every change of architecture, and it is the part of this course that will still be true in twenty years.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sculley et al. — Hidden Technical Debt in Machine Learning Systems (free)	The deployment reality of §1 and §2 , from Google. Feedback loops, entanglement, and why the model is the small part.
Google — Rules of Machine Learning, and the Model Cards paper (free)	The operational practices of §1, and the documentation artifact of §3.
Kleinberg, Mullainathan & Raghavan — Inherent Trade-Offs in the Fair Determination of Risk Scores (free)	The §3 impossibility theorem. Read the statement even if you skip the proof — it reframes the whole debate.

Resource	What it covers for this module
Barocas, Hardt & Narayanan — Fairness and Machine Learning (free book)	Free in full , and the best treatment of §3 as an engineering and measurement problem rather than a slogan.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Simulate covariate shift** on your project and measure how far performance drops before any input monitor would fire.
- 2 Implement feature-distribution monitoring with a population stability index and set a change-based alert.
- 3 **Simulate concept drift** and show that input monitoring does not detect it.
- 4 **Simulate a feedback loop**: let the model's decisions determine which outcomes you observe, retrain, and plot the drift over ten rounds.
- 5 **Add a randomised control slice** to that simulation and show it recovers an unbiased estimate.
- 6 **Measure all three fairness criteria** on a model with unequal base rates and confirm they cannot be satisfied together.
- 7 Remove a protected attribute and **demonstrate that a proxy reconstructs it**.
- 8 Report your project's metrics by subgroup, with counts, and name the worst slice.
- 9 **Write a model card** for your project: intended use, measured performance by slice, disparities, failure modes, retrain trigger.
- 10 **Project 2 is now due**. Submit the model, the evaluation against the committed protocol, the slice breakdown, the shift experiment, the error analysis, and the deployment statement.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name three kinds of drift and say which is detectable without labels.
- 2 Why monitor inputs rather than performance?
- 3 Why alert on change rather than threshold?
- 4 Describe a feedback loop and say why the usual diagnostics do not fire.
- 5 Why is a randomised control slice the mitigation that works?
- 6 State the three fairness criteria and the incompatibility result.
- 7 Why does removing the protected attribute fail?

8 Give four honest claims and one thing you cannot say.